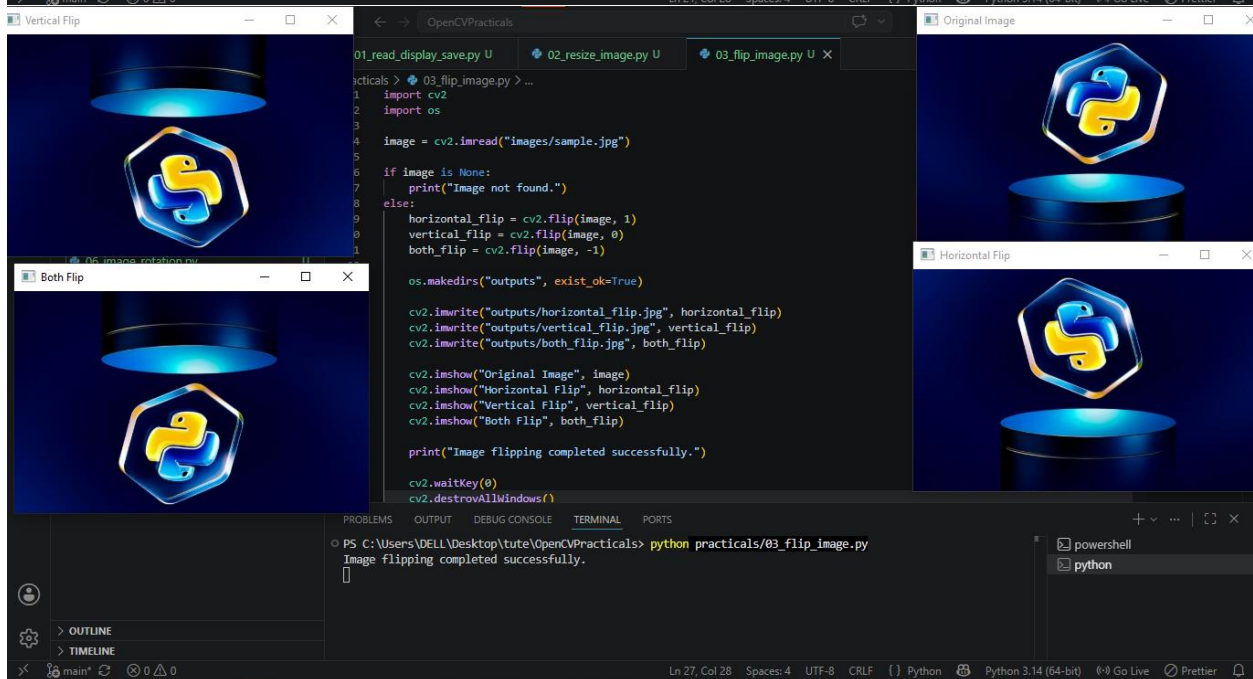
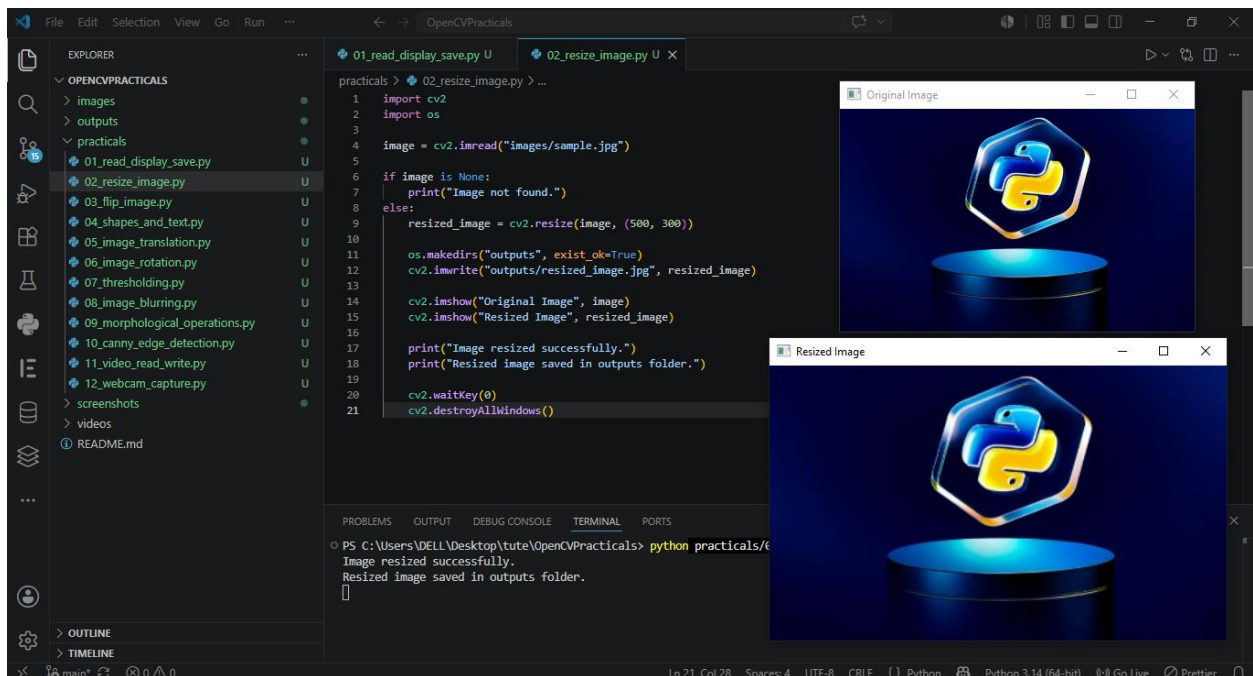




OpenCVPracticals

practicals > 04_shapes_and_text.py

```
1 import cv2
2 import os
3
4 image = cv2.imread("images/sample.jpg")
5
6 if image is None:
7     print("Image not found.")
8 else:
9     output = image.copy()
10
11     # Draw a line
12     cv2.line(output, (50, 50), (300, 50), (255, 0, 0), 4)
13
14     # Draw a rectangle
15     cv2.rectangle(output, (50, 100), (300, 250), (0, 255, 0), 4)
16
17     # Draw a circle
18     cv2.circle(output, (450, 170), 70, (0, 0, 255), 4)
19
20     # Add text
21     cv2.putText(
22         output,
23         "OpenCV Practical",
24         (50, 280),
25         cv2.FONT_HERSHEY_SIMPLEX,
26         1,
27         (255, 255, 255),
28     )
29
30 os.makedirs("outputs", exist_ok=True)
31 cv2.imwrite("outputs/shapes_and_text.jpg", output)
32
33 cv2.imshow("Original Image", image)
34 cv2.imshow("Shapes and Text", output)
35
36 print("Shapes and text added successfully.")
37 print("Output saved in outputs/shapes_and_text.jpg")
38
39 cv2.waitKey(0)
40 cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals/04_shapes_and_text.py
Shapes and text added successfully.
Output saved in outputs/shapes_and_text.jpg

Search

OpenCVPracticals

practicals > 05_image_translation.py

```
1 import cv2
2 import numpy as np
3 import os
4
5 # Read image
6 image = cv2.imread("images/sample.jpg")
7
8 if image is None:
9     print("Image not found.")
10 else:
11     rows, cols = image.shape[:2]
12
13     # Translation Matrix
14     tx = 100 # Shift right
15     ty = 50 # Shift down
16
17     translation_matrix = np.float32([
18         [1, 0, tx],
19         [0, 1, ty]
20     ])
21
22     translated = cv2.warpAffine(image, translation_matrix, (cols, rows))
23
24     os.makedirs("outputs", exist_ok=True)
25     cv2.imwrite("outputs/translated_image.jpg", translated)
26
27     cv2.imshow("Original Image", image)
28     cv2.imshow("Translated Image", translated)
29
30     print("Image translated successfully.")
31     print("Output saved in outputs/translated_image.jpg")
32
33     cv2.waitKey(0)
34     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals/04_shapes_and_text.py
Shapes and text added successfully.
Output saved in outputs/shapes_and_text.jpg
PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals/05_image_translation.py
Image translated successfully.
Output saved in outputs/translated_image.jpg

Original Image

Shapes and Text

Translated Image

OpenCVPracticals

01_read_display_save.py U 02_resize_image.py U 03_flip_image.py U 04_shapes_and_text.py U 05_image_translation.py U 06_image_rotation.py U

```
practicals > 06_image_rotation.py > ...
4 # Read image
5 image = cv2.imread("images/sample.jpg")
6
7 if image is None:
8     print("Image not found.")
9 else:
10     rows, cols = image.shape[:2]
11
12     # Rotation Matrix (45 degrees)
13     rotation_matrix = cv2.getRotationMatrix2D((cols / 2, rows / 2), 45, 1)
14
15     # Rotate image
16     rotated = cv2.warpAffine(image, rotation_matrix, (cols, rows))
17
18     # Create output folder
19     os.makedirs("outputs", exist_ok=True)
20
21     # Save rotated image
22     cv2.imwrite("outputs/rotated_image.jpg", rotated)
23
24     # Display images
25     cv2.imshow("Original Image", image)
26     cv2.imshow("Rotated Image", rotated)
27
28     print("Image rotated successfully.")
29     print("Output saved in outputs/rotated_image.jpg")
30
31     cv2.waitKey(0)
32     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals\06_image_rotation.py
Image rotated successfully.
Output saved in outputs/rotated_image.jpg

Original Image

Rotated Image

01_read_display_save.py U 02_resize_image.py U 03_flip_image.py U 04_shapes_and_text.py U 05_image_translation.py U 06_image_rotation.py U 07_thresholding.py U

```
practicals > 07_thresholding.py > ...
9 # Read image
10 image = cv2.imread("images/sample.jpg")
11
12 if image is None:
13     print("Image not found.")
14 else:
15     # Convert image to grayscale
16     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
17
18     # Apply binary threshold
19     threshold_value, binary_image = cv2.threshold(
20         gray_image,
21         127,
22         255,
23         cv2.THRESH_BINARY
24     )
25
26     # Create output folder
27     os.makedirs("outputs", exist_ok=True)
28
29     # Save output
30     cv2.imwrite("outputs/binary_threshold.jpg", binary_image)
31
32     # Display images
33     cv2.imshow("Original Image", image)
34     cv2.imshow("Grayscale Image", gray_image)
35     cv2.imshow("Binary Threshold", binary_image)
36
37     print("Binary thresholding completed successfully.")
38     print("Threshold value:", threshold_value)
39     print("Output saved in outputs/binary_threshold.jpg")
40
41     cv2.waitKey(0)
42     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals\06_image_rotation.py
Image rotated successfully.
Output saved in outputs/rotated_image.jpg

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals\07_thresholding.py
Binary thresholding completed successfully.
Threshold value: 127.0
Output saved in outputs/binary_threshold.jpg

Original Image

Grayscale Image

Binary Threshold

main

Ln 32, Col 28 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live Prettier

37°C ENG 6:22 PM

File Edit Selection View Go Run ... OpenCVPracticals

practicals > 01_read_display_save.py U X

```
1 import cv2
2 import os
3
4 # Read image
5 image = cv2.imread("images/sample.jpg")
6
7 if image is None:
8     print("Image not found.")
9 else:
10     # Apply Gaussian Blur
11     gaussian_blur = cv2.GaussianBlur(image, (11, 11), 0)
12
13     # Apply Median Blur
14     median_blur = cv2.medianBlur(image, 11)
15
16     # Create output folder
17     os.makedirs("outputs", exist_ok=True)
18
19     # Save output images
20     cv2.imwrite("outputs/gaussian_blur.jpg", gaussian_blur)
21     cv2.imwrite("outputs/median_blur.jpg", median_blur)
22
23     # Display images
24     cv2.imshow("Original Image", image)
25     cv2.imshow("Gaussian Blur", gaussian_blur)
26     cv2.imshow("Median Blur", median_blur)
27
28     print("Image blurring completed successfully.")
29     print("Gaussian Blur saved as outputs/gaussian_blur.jpg")
30     print("Median Blur saved as outputs/median_blur.jpg")
31
32     cv2.waitKey(0)
33     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals/07_thresholding.py
Output saved in outputs/binary_threshold.jpg
PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals/08_image_blurring.py
Image blurring completed successfully.
Gaussian Blur saved as outputs/gaussian_blur.jpg
Median Blur saved as outputs/median_blur.jpg

EXPLORER

- OPENCVPRACTICALS
 - images
 - outputs
 - practicals
 - 01_read_display_save.py U
 - 02_resize_image.py U
 - 03_flip_image.py U
 - 04_shapes_and_text.py U
 - 05_image_translation.py U
 - 06_image_rotation.py U
 - 07_thresholding.py U
 - 08_image_blurring.py U
 - 09_morphological_operations.py U
 - 10_canny_edge_detection.py U
 - 11_video_read_write.py U
 - 12_webcam_capture.py U
 - screenshots
 - videos
 - README.md

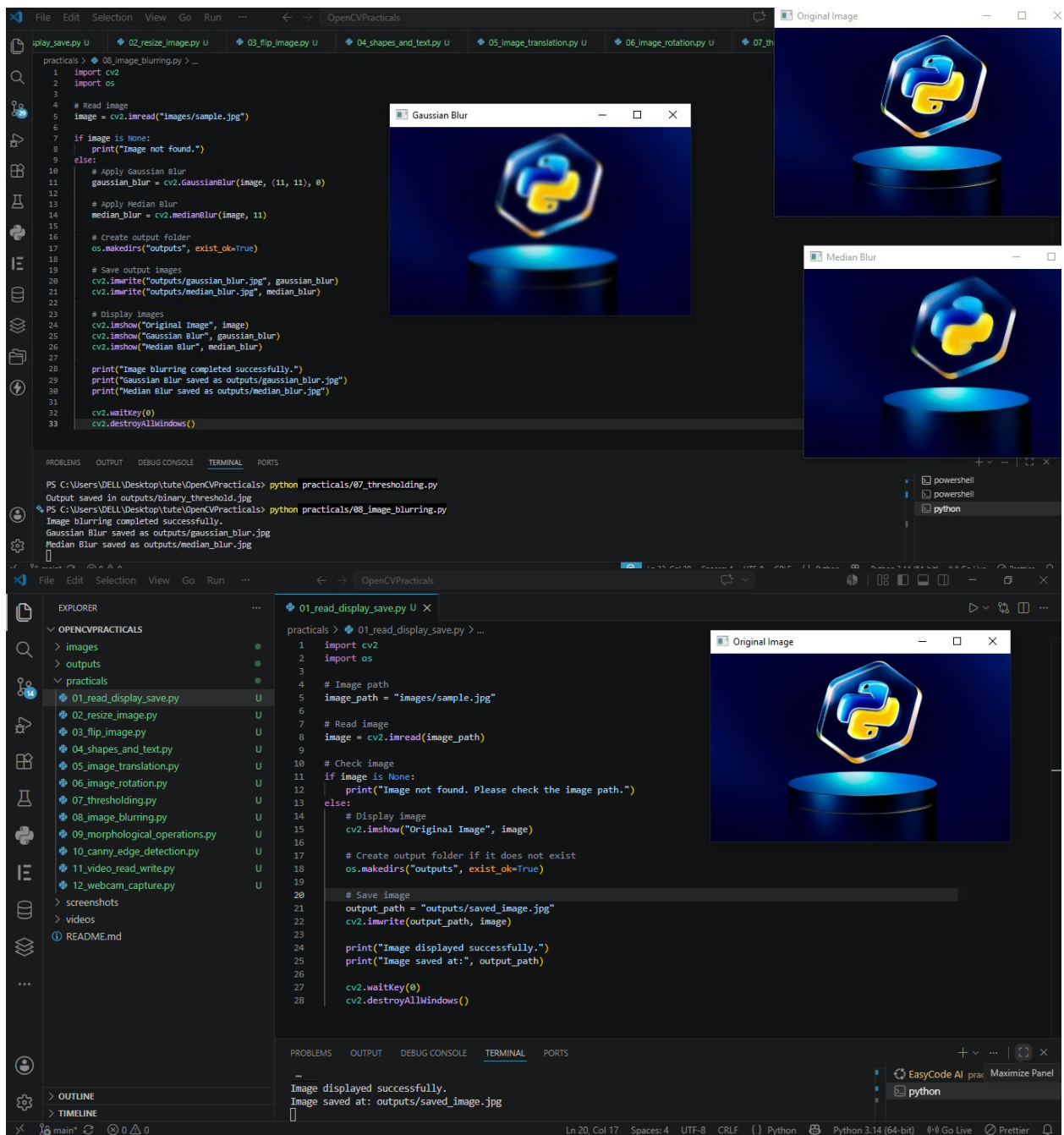
01_read_display_save.py U X

```
1 import cv2
2 import os
3
4 # Image path
5 image_path = "images/sample.jpg"
6
7 # Read image
8 image = cv2.imread(image_path)
9
10 # Check image
11 if image is None:
12     print("Image not found. Please check the image path.")
13 else:
14     # Display image
15     cv2.imshow("Original Image", image)
16
17     # Create output folder if it does not exist
18     os.makedirs("outputs", exist_ok=True)
19
20     # Save image
21     output_path = "outputs/saved_image.jpg"
22     cv2.imwrite(output_path, image)
23
24     print("Image displayed successfully.")
25     print("Image saved at:", output_path)
26
27     cv2.waitKey(0)
28     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Image displayed successfully.
Image saved at: outputs/saved_image.jpg

main* 0 Ln 20, Col 17 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) 6-8 Go Live Prettier



The image shows a screenshot of a Python IDE (likely VS Code) with a dark theme. The main editor window displays a Python script for image blurring using OpenCV. The script reads an image from 'images/sample.jpg', applies Gaussian and Median blurring, saves the results to 'outputs', and displays them. The script is executed, and the output is visible in the terminal. To the right of the editor, there are two windows showing the original image and the blurred results. The Explorer panel on the left shows the project structure, including a 'practicals' folder with various Python scripts. The bottom status bar shows the current file is 'main*' at line 20, column 17, with a UTF-8 encoding and CRLF line endings. The status bar also indicates the Python version is 3.14 (64-bit) and shows icons for Go Live and Prettier.

OpenCVPracticals

practicals > 09_morphological_operations.py > ...

```
1 import cv2
2 import os
3
4 # Read image
5 image = cv2.imread("images/sample.jpg")
6
7 if image is None:
8     print("Image not found.")
9 else:
10     # Convert to grayscale
11     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
12
13     # Create kernel
14     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15))
15
16     # Morphological operations
17     top_hat = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)
18     black_hat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)
19
20     # Create output folder
21     os.makedirs("outputs", exist_ok=True)
22
23     # Save output images
24     cv2.imwrite("outputs/top_hat.jpg", top_hat)
25     cv2.imwrite("outputs/black_hat.jpg", black_hat)
26
27     # Display images
28     cv2.imshow("Original Image", image)
29     cv2.imshow("Top Hat", top_hat)
30     cv2.imshow("Black Hat", black_hat)
31
32     print("Morphological operations completed successfully.")
33     print("Top Hat image saved as outputs/top_hat.jpg")
34     print("Black Hat image saved as outputs/black_hat.jpg")
35
36     cv2.waitKey(0)
37     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals\09_morphological_operations.py
Top Hat image saved as outputs/top_hat.jpg
Black Hat image saved as outputs/black_hat.jpg

OpenCVPracticals

practicals > 10_canny_edge_detection.py > ...

```
1 import cv2
2 import os
3
4 # Read image
5 image = cv2.imread("images/sample.jpg")
6
7 if image is None:
8     print("Image not found.")
9 else:
10     # Convert to grayscale
11     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
12
13     # Apply Canny Edge Detection
14     edges = cv2.Canny(gray, 100, 200)
15
16     # Create output folder
17     os.makedirs("outputs", exist_ok=True)
18
19     # Save output
20     cv2.imwrite("outputs/canny_edges.jpg", edges)
21
22     # Display images
23     cv2.imshow("Original Image", image)
24     cv2.imshow("Canny Edge Detection", edges)
25
26     print("Canny Edge Detection completed successfully.")
27     print("Output saved in outputs/canny_edges.jpg")
28
29     cv2.waitKey(0)
30     cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\tute\OpenCVPracticals> python practicals\10_canny_edge_detection.py
Canny Edge Detection completed successfully.
Output saved in outputs/canny_edges.jpg

Original Image

Top Hat

Black Hat

Canny Edge Detection

